

CS 671 | Deep Learning

Variational Autoencoders

Evidence Lower Bound (ELBO) — Complete Derivation & Lecture Notes

Motivation | Full Proof (Two Methods) | Reparameterization Trick | Implementation

Dr. Aditya Nigam | IIT Mandi

CS 671 — Deep Generative Models

0. Overview & Learning Objectives

What is a Variational Autoencoder?

A Variational Autoencoder (VAE) is a generative latent-variable model that learns a structured probabilistic latent space. Unlike standard autoencoders, the encoder outputs the parameters of a probability distribution — not a single vector. Training is performed by maximizing the Evidence Lower Bound (ELBO), a principled surrogate for the intractable log-likelihood.

After completing these notes you will be able to:

- **State** the generative model and the inference problem in a VAE.
- **Derive** the ELBO from scratch using two independent methods.
- **Interpret** each term of the ELBO: reconstruction loss and KL regularization.
- **Explain** why direct maximization of $\log p(x)$ is intractable.
- **Apply** the reparameterization trick to enable gradient-based optimization.
- **Connect** the ELBO objective to the encoder-decoder architecture.

1. The Generative Model

1.1 Probabilistic Setup

We have a dataset $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$ of i.i.d. observations. Each x is assumed generated from an unobserved latent variable z via a two-step process:

1. **Sample latent:** $z \sim p(z)$ — draw from the prior distribution over latent codes.
2. **Sample observation:** $x \sim p(x | z; \theta)$ — the decoder generates data conditioned on the latent code.

$$\text{Joint distribution: } p(x, z; \theta) = p(x | z; \theta) \cdot p(z)$$

$$\text{Marginal likelihood: } p(x; \theta) = \int p(x | z; \theta) p(z) dz$$

The marginal integrates over all possible latent codes — computationally intractable

Intuition

Think of handwritten digit images. The latent variable z encodes abstract concepts like 'which digit', 'stroke thickness', 'slant angle'. Given z , the decoder generates pixel values. The posterior $p(z|x)$ answers: 'given this image, which latent concepts likely produced it?'

1.2 The Intractability Problem

Our goal is to maximize the log-likelihood of the data:

$$\log p(x; \theta) = \log \int p(x | z; \theta) p(z) dz$$

This integral is intractable for three reasons:

- **High dimensionality:** z is typically hundreds of dimensions — numerical integration is computationally impossible.
- **Non-linear decoder:** $p(x|z; \theta)$ is a neural network, making the integral analytically insolvable.
- **Circular dependency:** Computing $p(z|x)$ via Bayes' theorem requires $p(x)$, which is exactly what we want to compute.

Core Challenge

Since $p(x)$ is intractable to compute or maximize directly, we derive a tractable lower bound — the ELBO — and maximize that instead. Maximizing a tight lower bound is a principled surrogate for maximizing the true log-likelihood.

2. Variational Inference

2.1 Introducing the Approximate Posterior

Variational inference replaces the intractable true posterior $p(z|x)$ with a tractable parametric family $q(z|x; \varphi)$. In a VAE, the encoder network outputs:

$$\text{Encoder output: } q(z | x; \varphi) = \mathcal{N}(z; \mu(x, \varphi), \text{diag}(\sigma^2(x, \varphi)))$$

$$\text{Architecture: } x \longrightarrow [\text{Neural Net } \varphi] \longrightarrow \mu(x, \varphi), \sigma^2(x, \varphi)$$

The encoder maps each input x to the mean and diagonal variance of the approximate posterior

2.2 KL Divergence — Measuring Approximation Quality

We measure how well q approximates the true posterior $p(z|x)$ using the KL divergence:

$$\text{KL}[q(z|x; \varphi) \parallel p(z|x; \theta)] = \mathbb{E}_{\{q\}}[\log q(z|x; \varphi) - \log p(z|x; \theta)]$$

$$\geq 0 \quad (\text{Gibbs' inequality — always non-negative})$$

KL divergence equals 0 if and only if $q = p$ almost everywhere

Applying Bayes' theorem to expand $p(z|x; \theta)$:

$$p(z|x; \theta) = p(x|z; \theta) \cdot p(z) / p(x; \theta) \quad [\text{Bayes' theorem}]$$

3. ELBO Derivation — Method 1: KL Decomposition

This is the most direct derivation. We start from the KL divergence between the approximate posterior and the true posterior, then algebraically isolate $\log p(x)$.

Step 1

Define KL

$$\text{KL}[q(z|x;\varphi) \parallel p(z|x;\theta)] = \mathbb{E}_q[\log q(z|x;\varphi) - \log p(z|x;\theta)]$$

Definition of KL divergence. Both terms are log-probabilities. The expectation is taken under $q(z|x;\varphi)$.

Step 2

Apply Bayes

$$\log p(z|x;\theta) = \log p(x|z;\theta) + \log p(z) - \log p(x;\theta)$$

Bayes' theorem: $p(z|x) = p(x|z)p(z)/p(x)$. Take the logarithm of both sides and rearrange.

Step 3

Substitute
Bayes into KL

$$\text{KL} = \mathbb{E}_q[\log q(z|x;\varphi) - \log p(x|z;\theta) - \log p(z) + \log p(x;\theta)]$$

Substitute Step 2 into Step 1. Distribute the expectation linearly over each additive log-term.

Step 4

Pull Out log
 $p(x)$

$$\text{KL} = \mathbb{E}_q[\log q(z|x;\varphi) - \log p(x|z;\theta) - \log p(z)] + \log p(x;\theta)$$

$\log p(x;\theta)$ does not depend on z , so $\mathbb{E}_q[\log p(x;\theta)] = \log p(x;\theta)$. Extract it from the expectation.

Step 5
 Rearrange for
 ELBO

$$\log p(x; \theta) = \text{KL}[q \| p] + \mathbb{E}_q[\log p(x|z; \theta) + \log p(z) - \log q(z|x; \varphi)]$$

Rearrange. The right-hand expectation is the ELBO. Since $\text{KL} \geq 0$, we have $\text{ELBO} \leq \log p(x)$.

Step 6
 Identify the
 ELBO

$$\log p(x; \theta) = \text{KL}[q(z|x; \varphi) \| p(z|x; \theta)] + \text{ELBO}(\theta, \varphi; x)$$

Name the middle term 'ELBO'. This is the fundamental identity of variational inference.

KEY IDENTITY:

$$\log p(x; \theta) = \text{KL}[q(z|x; \varphi) \| p(z|x; \theta)] + \text{ELBO}(\theta, \varphi; x)$$

$$\text{Since } \text{KL} \geq 0: \quad \log p(x; \theta) \geq \text{ELBO}(\theta, \varphi; x)$$

$$\text{Gap} = 0 \iff q(z|x; \varphi) = p(z|x; \theta) \quad (\text{exact posterior recovery})$$

The ELBO is a lower bound on $\log p(x)$. Maximizing the ELBO pushes $\log p(x)$ upward.

4. ELBO Derivation — Method 2: Jensen's Inequality

This approach derives the ELBO directly from $\log p(x)$ by introducing q as an importance weighting distribution and applying Jensen's inequality.

Step 1
 Start from
 $\log p(x)$

$$\log p(x; \theta) = \log \int p(x, z; \theta) \, dz$$

Marginalise the joint distribution $p(x, z; \theta)$ over the latent variable z by integrating it out.

Step 2
 Multiply by
 q/q (trick)

$$= \log \int q(z|x; \varphi) \cdot [p(x, z; \theta) / q(z|x; \varphi)] \, dz$$

Multiply and divide inside the integral by $q(z|x; \varphi)$. This is the importance weighting identity.

Step 3
 Write as
 Expectation
 under q

$$= \log \mathbb{E}_{\{q(z|x; \varphi)\}} [p(x, z; \theta) / q(z|x; \varphi)]$$

Recognise the integral as the expectation $\mathbb{E}_q[\cdot]$. The integrand is an importance weight.

Step 4

Apply Jensen's Inequality

$$\geq \mathbb{E}_{\mathbf{q}}\{q(\mathbf{z}|\mathbf{x};\varphi)\} \left[\log \left(\frac{p(\mathbf{x}, \mathbf{z}; \theta)}{q(\mathbf{z}|\mathbf{x}; \varphi)} \right) \right]$$

Jensen's inequality: $\log$ is concave, so $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$. Move $\log$ inside the expectation.

Step 5

Expand the Log Ratio

$$= \mathbb{E}_{\mathbf{q}}[\log p(\mathbf{x}, \mathbf{z}; \theta) - \log q(\mathbf{z}|\mathbf{x}; \varphi)]$$

$\log(A/B) = \log A - \log B$. Distribute the expectation linearly over each term.

Step 6

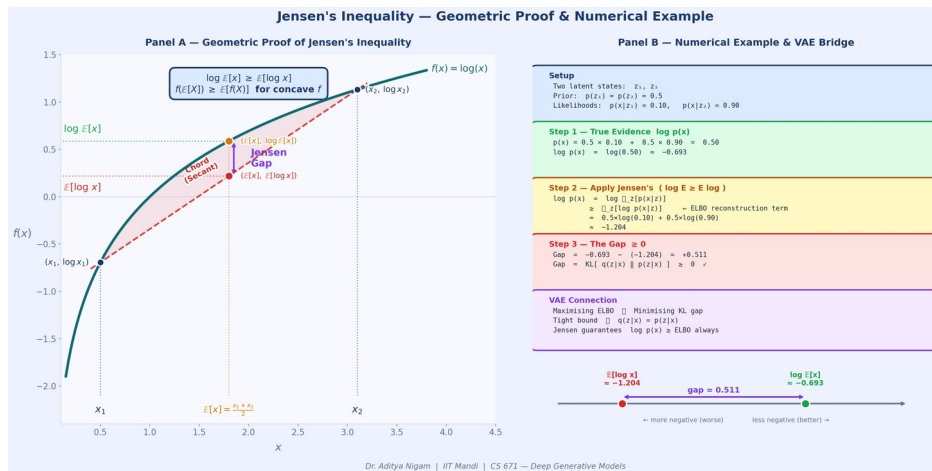
Expand the Joint

$$= \mathbb{E}_{\mathbf{q}}[\log p(\mathbf{x}|\mathbf{z}; \theta) + \log p(\mathbf{z}) - \log q(\mathbf{z}|\mathbf{x}; \varphi)]$$

Chain rule: $\log p(\mathbf{x}, \mathbf{z}; \theta) = \log p(\mathbf{x}|\mathbf{z}; \theta) + \log p(\mathbf{z})$. This is the final ELBO expression.

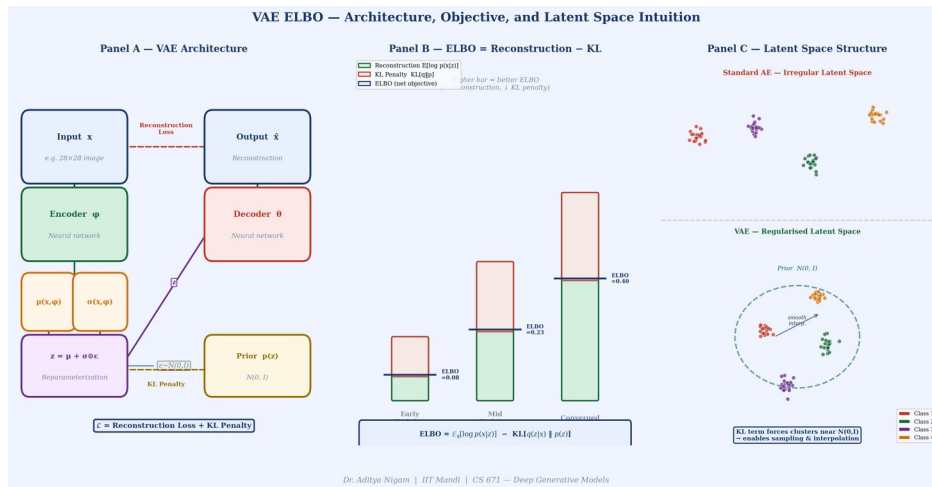
Jensen's Inequality — Reminder

For any concave function f and random variable X : $f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$. Since $\log$ is strictly concave, the inequality is tight if and only if X is constant — i.e., $q(\mathbf{z}|\mathbf{x}; \varphi) = p(\mathbf{z}|\mathbf{x}; \theta)$. This confirms the ELBO equals $\log p(\mathbf{x})$ exactly when the encoder matches the true posterior.

Figure 1 — Jensen's Inequality: Geometric Proof & Numerical Example

Panel A: The concave $\log$ curve lies above every chord (red dashed). The gap between $\log \mathbb{E}[x]$ (on the curve, orange) and $\mathbb{E}[\log x]$ (on the chord, red) is the Jensen gap, which equals $KL[q \parallel p(\mathbf{z}|\mathbf{x})]$. Panel B: Concrete walk-through with two latent states showing gap = 0.511.

5. The Full ELBO Expression & Its Two Terms**Figure 2 — VAE Architecture, ELBO Training Progression, and Latent Space Regularisation**



Panel A: Data flows through encoder $\rightarrow$ reparameterization $\rightarrow$ decoder. Panel B: ELBO grows as training progresses — reconstruction improves while KL stabilises. Panel C: KL term forces latent clusters inside $\mathcal{N}(0, \mathbf{I})$, enabling smooth generation.

$$\text{ELBO}(\theta, \varphi; x) = \mathbb{E}_{\{q(z|x; \varphi)\}} [\log p(x|z; \theta)] - \text{KL}[q(z|x; \varphi) \parallel p(z)]$$

Reconstruction Term

KL Regularization

5.1 Term 1 — Reconstruction Loss

$$\text{Reconstruction: } \mathbb{E}_{\{q(z|x; \varphi)\}} [\log p(x|z; \theta)]$$

Expected log-probability of x under the decoder, when z is drawn from the encoder

This term measures how faithfully the decoder reconstructs x from latent codes sampled from the encoder. In practice:

- **Binary data (e.g., binarized images):** $\log p(x|z)$ is the negative binary cross-entropy (Bernoulli likelihood).
- **Continuous data:** $\log p(x|z)$ is the negative mean squared error (Gaussian likelihood with fixed variance).
- **Interpretation:** Maximizing this term makes the decoder produce outputs that explain the observed data.

5.2 Term 2 — KL Regularization

$$\text{KL}[q(z|x; \varphi) \parallel p(z)]$$

With $q = \mathcal{N}(\mu, \sigma^2 \mathbf{I})$ and $p = \mathcal{N}(0, \mathbf{I})$ (closed-form):

$$= \frac{1}{2} \sum_j [\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1]$$

Sum is over each latent dimension j . Computed analytically — no sampling required.

This term measures how far the encoder distribution deviates from the prior. It acts as a regularizer that:

- **Prevents overfitting:** Forces encoder posteriors to stay near $\mathcal{N}(0, \mathbf{I})$, preventing the encoder from memorizing inputs.
- **Enables generation:** A trained VAE can sample $z \sim \mathcal{N}(0, \mathbf{I})$ and decode novel samples, because the prior is meaningful.
- **Promotes structure:** Diagonal covariance assumption encourages independent, interpretable latent dimensions.

Derivation of Closed-Form KL

Let $q = \mathcal{N}(\mu, \sigma^2 \mathbf{I})$ and $p = \mathcal{N}(0, \mathbf{I})$ over d dimensions. $\text{KL}[q||p] = \mathbb{E}_q[\log q(z) - \log p(z)]$
 After computing expectations (using $\mathbb{E}[z_j] = \mu_j$, $\text{Var}[z_j] = \sigma_j^2$): $\text{KL}[q||p] = \frac{1}{2} \sum_j [\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1]$ (standard closed-form in all VAE code)

6. Training Objective & Bound Tightness

6.1 The ELBO as an Optimization Objective

We train the VAE to **maximize the ELBO** jointly over encoder parameters φ and decoder parameters θ :

$$\max_{\{\theta, \varphi\}} \text{ELBO}(\theta, \varphi; x)$$

$$= \max_{\{\theta, \varphi\}} \mathbb{E}_q[\log p(x|z; \theta)] - \text{KL}[q(z|x; \varphi) || p(z)]$$

Equivalently: minimise $\mathcal{L} = -\mathbb{E}_q[\log p(x|z; \theta)] + \text{KL}[q(z|x; \varphi) || p(z)]$

Gradient flow: Gradients flow back through both terms: $\partial \text{ELBO} / \partial \theta$ improves the decoder; $\partial \text{ELBO} / \partial \varphi$ improves the encoder — but the latter requires the reparameterization trick (Section 7).

6.2 The Gap and Bound Tightness

$$\log p(x; \theta) - \text{ELBO}(\theta, \varphi; x) = \text{KL}[q(z|x; \varphi) || p(z|x; \theta)] \geq 0$$

$$\text{Gap closes} \iff q(z|x; \varphi) = p(z|x; \theta) \quad (\text{encoder} = \text{true posterior})$$

As training progresses, a better encoder tightens the bound around $\log p(x)$

7. The Reparameterization Trick

7.1 Why Sampling Breaks Backpropagation

To evaluate $\mathbb{E}_q[\log p(x|z;\theta)]$, we sample $z \sim q(z|x;\varphi)$. But the gradient $\partial/\partial\varphi \mathbb{E}_q[\dots]$ requires differentiating through a stochastic sampling node, which standard autograd cannot do.

7.2 The Reparameterization

We rewrite z as a deterministic function of a fixed noise variable ε :

Step 1 — Sample fixed noise: $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$

Step 2 — Deterministic transform: $z = \mu(x, \varphi) + \sigma(x, \varphi) \odot \varepsilon$

Result: $z \sim \mathcal{N}(\mu(x, \varphi), \text{diag}(\sigma^2(x, \varphi))) \checkmark$

MC gradient estimate (L samples):

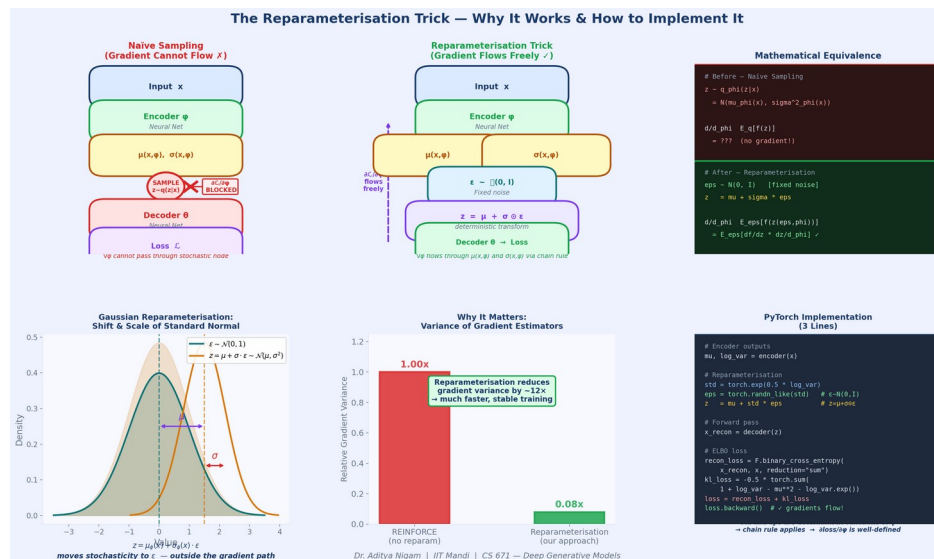
$$\partial \text{ELBO} / \partial \varphi \approx (1/L) \sum_1 \partial / \partial \varphi \log p(x | \mu + \sigma \odot \varepsilon^\omega; \theta)$$

In practice $L = 1$ works well since gradients are averaged over large mini-batches

Why This Works

The trick moves stochasticity to an input ε that does **not** depend on φ . Backpropagation can then propagate gradients through the deterministic functions $\mu(x, \varphi)$ and $\sigma(x, \varphi)$ via the chain rule. Without this, we would need high-variance REINFORCE-style estimators, which are impractical for continuous latent spaces.

Figure 3 — The Reparameterisation Trick: Comparison, Intuition & Implementation



Top-left: Naïve sampling blocks the gradient at the stochastic node. Top-centre: Reparameterisation moves ϵ outside the computation graph so $\partial \mathcal{L} / \partial \phi$ flows freely through μ and σ . Top-right: Before/After mathematical comparison. Bottom-left: Gaussian shift & scale intuition. Bottom-centre: Variance reduction ($\sim 12\times$) over REINFORCE. Bottom-right: PyTorch implementation (3 key lines).

8. The Complete VAE Training Algorithm

Putting everything together:

3. **Sample mini-batch:** $\{x^{(1)}, \dots, x^{(M)}\} \sim \mathcal{D}$
4. **Encode:** Pass each $x^{(i)}$ through encoder to get $\mu^{(i)}, \sigma^{2(i)}$
5. **Reparameterize:** Sample $\epsilon^{(i)} \sim \mathcal{N}(0, \mathbf{I})$, compute $z^{(i)} = \mu^{(i)} + \sigma^{(i)} \odot \epsilon^{(i)}$
6. **Decode:** Pass $z^{(i)}$ through decoder to get reconstruction parameters
7. **Compute loss:** $\mathcal{L} = -\mathbb{E}_q[\log p(x|z;\theta)] + \text{KL}[q(z|x;\phi) \parallel p(z)]$
8. **Backpropagate & update:** Compute gradients $\partial \mathcal{L} / \partial \theta, \partial \mathcal{L} / \partial \phi$ and apply gradient descent

VAE Loss (per sample, to minimize):

$$\mathcal{L}(\theta, \phi; x) = -\mathbb{E}_q[\log p(x|z;\theta)] + \text{KL}[q(z|x;\phi) \parallel p(z)]$$

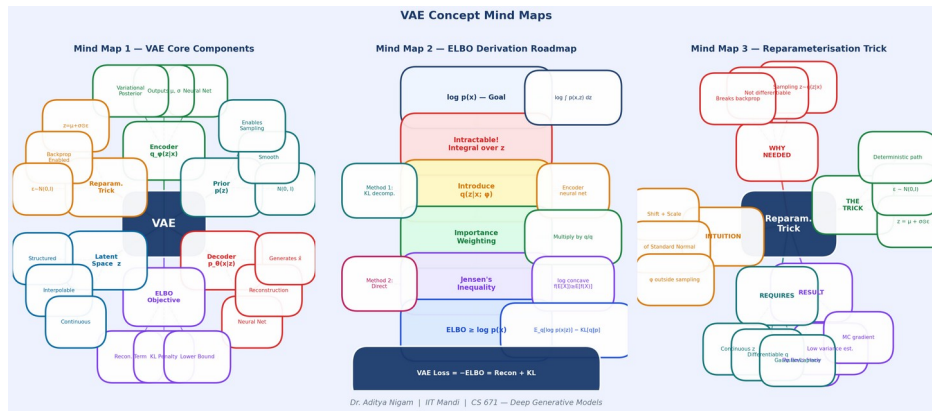
$$= \text{Reconstruction Loss} + \text{KL Penalty}$$

$$\text{In code: } \mathcal{L} = \text{BCE}(\hat{x}, x) + \frac{1}{2} \cdot \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$$

9. Concept Mind Maps

The following mind maps provide a bird's-eye view of the key concepts, their relationships, and the derivation roadmap.

Figure 4 — VAE Concept Mind Maps: Components, Derivation Roadmap & Reparameterisation



Mind Map 1: All six core VAE components and their sub-concepts. Mind Map 2: The ELBO derivation as a vertical flowchart — from the intractable $\log p(x)$ goal down to the final loss. Mind Map 3: The reparameterisation trick broken into Why Needed, The Trick, Result, Intuition, and Requirements.

10. Symbol Reference Table

Symbol	Meaning	Type / Space
x	Observed data (image, audio, etc.)	Random variable, $x \in \mathcal{X}$
z	Latent variable (hidden code)	Random variable, $z \in \mathbb{R}^d$
$p(x)$	True marginal / evidence (intractable)	Scalar probability
$p(z)$	Prior over latents	$\mathcal{N}(0, I)$, fixed
$p(x z; \theta)$	Generative model / decoder	Parametric neural net
$p(z x; \theta)$	True posterior (intractable)	Target of approximation
$q(z x; \phi)$	Approximate posterior / encoder	Parametric neural net
θ	Decoder network parameters	Learned via gradient descent
ϕ	Encoder network parameters	Learned via gradient descent
$\mathbb{E}_{q[\cdot]}$	Expectation under $q(z x)$	Scalar
$\mathcal{N}(\mu, \sigma^2)$	Gaussian distribution with mean μ , variance σ^2	Probability distribution
$\text{KL}[q p]$	KL divergence (forward)	Non-negative scalar
$\mathcal{L}(\theta, \phi; x)$	Negative ELBO (loss to minimize)	Minimized during training
ϵ	Noise variable for reparameterization	$\mathcal{N}(0, I)$, fixed
$\odot$	Element-wise (Hadamard) product	Vector operation

10. Common Pitfalls & Exam Tips

- **KL direction:** $\text{KL}[q\|p] \neq \text{KL}[p\|q]$. The ELBO uses the forward KL. Swapping the arguments gives a non-tractable expression.
- **Bound direction:** ELBO is a lower bound — we MAXIMIZE it. Never write $\text{ELBO} \geq \log p(x)$.
- **Jensen's direction:** log is concave: $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$. Moving log inside flips the inequality, giving the lower bound.
- **Reparameterization requires continuity:** The trick applies only to continuous latent variables. Discrete latents require Gumbel-Softmax or REINFORCE estimators.
- **Posterior collapse:** If the decoder is too powerful, it ignores z and the KL term collapses to zero. The encoder maps every input to the prior, encoding nothing useful.
- **β -VAE:** Multiplying the KL term by $\beta > 1$ encourages disentanglement at the cost of reconstruction quality.
- **Reconstruction loss choice:** Binary cross-entropy assumes Bernoulli outputs; MSE assumes Gaussian with fixed variance. Misspecifying this affects the trade-off.

11. Summary — ELBO at a Glance

THE ELBO IDENTITY:

$$\log p(x; \theta) = \text{KL}[q(z|x;\varphi) \parallel p(z|x;\theta)] + \text{ELBO}(\theta, \varphi; x)$$

$$\geq \text{ELBO} \quad (\text{since } \text{KL} \geq 0)$$

$$\text{ELBO} = \mathbb{E}_q[\log p(x|z;\theta)] - \text{KL}[q(z|x;\varphi) \parallel p(z)]$$

$$= \text{Reconstruction} - \text{KL Regularization}$$

$$\text{Reparameterization: } z = \mu(x, \varphi) + \sigma(x, \varphi) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

$$\text{KL (Gaussian): } \frac{1}{2} \sum_j [\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1]$$

The Big Picture

VAE training is a competition between two networks: the encoder $q(z|x;\varphi)$ infers latent codes, and the decoder $p(x|z;\theta)$ reconstructs data. The ELBO simultaneously rewards reconstruction quality and penalizes deviation of the encoder from the prior. Maximizing the ELBO is equivalent to minimizing the KL gap between our variational approximation and the true data distribution — a principled approach grounded in information theory and maximum likelihood estimation.

Reference: Kingma & Welling, 'Auto-Encoding Variational Bayes', ICLR 2014.